

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

Aim: To remember the important tasks and forget the less important tasks using LSTM

IDE: Google Colab

Theory:

Countless learning tasks require dealing with sequential data. Image captioning, speech synthesis, and music generation all require that models produce outputs consisting of sequences. In other domains, such as time series prediction, video analysis, and musical information retrieval, a model must learn from inputs that are sequences. These demands often arise simultaneously: tasks such as translating passages of text from one natural language to another, engaging in dialogue, or controlling a robot, demand that models both ingest and output sequentially-structured data.

Recurrent neural networks (RNNs) are deep learning models that capture the dynamics of sequences via *recurrent* connections, which can be thought of as cycles in the network of nodes. This might seem counterintuitive at first. After all, it is the feedforward nature of neural networks that makes the order of computation unambiguous. However, recurrent edges are defined in a precise way that ensures that no such ambiguity can arise. Recurrent neural networks are *unrolled* across time steps (or sequence steps), with the *same* underlying parameters applied at each step. While the standard connections are applied *synchronously* to propagate each layer's activations to the subsequent layer *at the same time step*, the recurrent connections are *dynamic*, passing information across adjacent time steps. As the unfolded view reveals, RNNs can be thought of as feedforward neural networks where each layer's parameters (both conventional and recurrent) are shared across time steps. Like neural networks more broadly, RNNs have a long discipline-spanning history, originating as models of the brain popularized by cognitive scientists and subsequently adopted as practical modeling tools employed by the machine learning community.

The name of LSTM refers to the analogy that a standard RNN has both "long-term memory" and "short-term memory". The connection weights and biases in the network change once per episode of training, analogous to how physiological changes in synaptic strengths store long-term memories; the activation patterns in the network change once per time-step, analogous to how the moment-to-moment change in electric firing patterns in the brain store short-term memories. The LSTM architecture aims to provide a short-term memory for RNN that can last thousands of timesteps, thus "long short-term memory".

A common LSTM unit is composed of a cell, an input gate, an output gate and a forget gate. The cell remembers values over arbitrary time intervals and the three gates regulate the flow of information into and out of the cell. LSTM networks are well-suited to classifying, processing and making predictions based on time series data, since there can be lags of unknown duration between important events in a time series. LSTMs were developed to deal with the vanishing gradient problem that can be encountered when training traditional RNNs. Relative

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

insensitivity to gap length is an advantage of LSTM over RNNs, hidden Markov models and other sequence learning methods in numerous applications.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Apply LSTM Model
5. Apply the training over the dataset to minimize the loss
6. Observe the cost function vs iterations learning curve

Program (Code):

```
#lab 5 code
```

```
# Step 1: Import Libraries
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

```
# Step 2: Generate Sample Sequential Data
```

```
data = np.sin(np.arange(0, 100, 0.1))
```

```
# Step 3: Prepare Dataset
```

```
scaler = MinMaxScaler()
data = scaler.fit_transform(data.reshape(-1,1))
```

```
X = []
```

```
y = []
```

```
time_step = 10
```

```
for i in range(len(data)-time_step-1):
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

```
X.append(data[i:(i+time_step), 0])
y.append(data[i+time_step, 0])
```

```
X = np.array(X)
y = np.array(y)
```

```
X = X.reshape(X.shape[0], X.shape[1], 1)
```

Step 4: Split Data

```
train_size = int(len(X)*0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
```

Step 5: Build LSTM Model

```
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(time_step,1)))
model.add(LSTM(50))
model.add(Dense(1))
```

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

Step 6: Train Model

```
history = model.fit(X_train, y_train, epochs=20,
validation_data=(X_test, y_test))
```

Step 7: Predictions

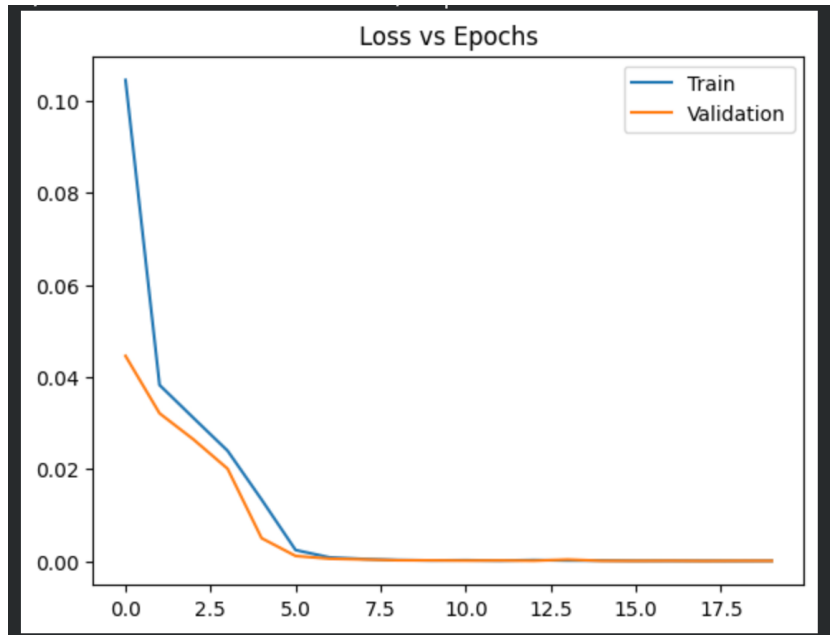
```
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
```

Step 8: Plot Accuracy (use loss as metric)

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss vs Epochs")
plt.legend(['Train', 'Validation'])
plt.show()
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

Results:



Observation and Result Analysis:

a. Nature of the dataset

Ans: the dataset consists of sequential time series data, values depend on previous time steps, patterns like trends and cycle are present, suitable for LSTM based prediction.

b. During Training Process

Ans: the model learns temporal dependencies in data, loss decreases gradually over epochs, validation loss may fluctuate initially, wieghts are updated to capture sequence patterns.

c. After the training Process

Ans: model predicts future values based on past sequences, predictions follow the trend of actual data, accuracy depends on sequence length and epochs, model generalizes well if overfitting is controlled.

d. Observation over the Learning Curve

Ans: loss decreases steadily indicating learning, validation curve shows model generalization, large gap indicates overfitting, smooth curves indicate stable training.

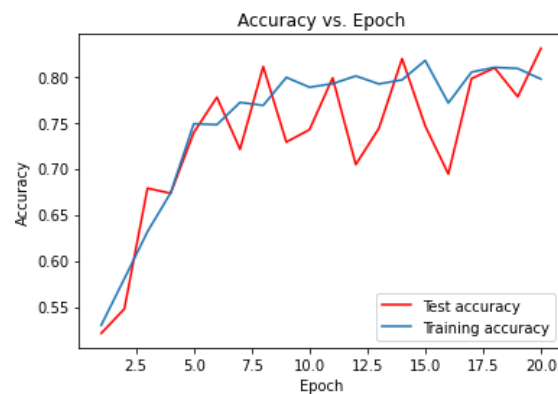
Post Lab Exercise:

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

a. What is the requirement of LSTM over RNN?

Ans: LSTM solves vanishing gradient problem in RNN, it can remember long term dependencies, uses gates to control information flow, more effective for long sequences.

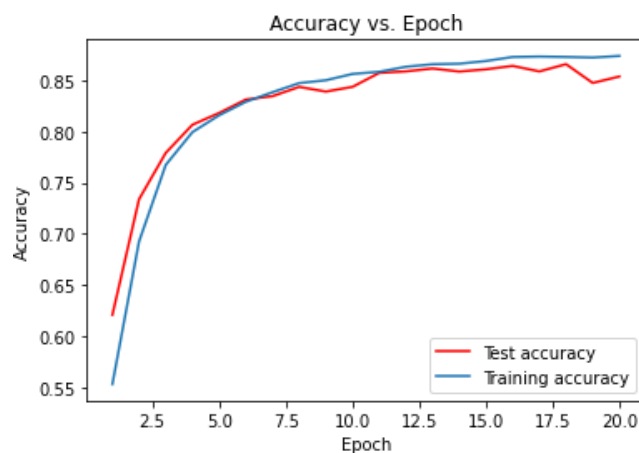
b. Interpretation of graph



What is the meaning of the above graph? How can the graph be smoothened?

Ans: training accuracy blue increases steadily showing learning, test accuracy red fluctuates a lot indicates overfitting, model is not generalizing well to unseen data, to smoothen use dropout, more data early stopping reduce learning rate.

c.



What is the meaning of the above graph? How can the graph be smoothened?

Ans: both training and test accuracy increase smoothly, very small gap between curves good generalization model is stable and well trained, smoothing achieved by regularization proper learning rate and tuning epochs.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

Post Lab Activity

Obtain the prediction value of the TCS stock for the historical data obtained from the link mentioned as: <https://in.investing.com/equities/tata-consultancy-services-historical-data>

Select the Date Range from (01-01-2020 to 31-03-2024). Select the price range to be as “DAILY” stock values. Train your LSTM model to obtain the predicted value as close as the actual value. Paste the screenshot of the output.

Code:

```
# Step 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

```
# Step 2: Load Dataset
df = pd.read_csv('TCS Historical Data.csv')
```

```
# Step 3: Clean Data
df.columns = df.columns.str.strip()
```

```
# Convert Price column to float
df['Price'] = df['Price'].str.replace(',', '').astype(float)
```

```
# Use only Price
data = df[['Price']].values
```

```
# Step 4: Normalize
scaler = MinMaxScaler()
data = scaler.fit_transform(data)
```

```
# Step 5: Create Sequences
X, y = [], []
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

```
time_step = 60
```

```
for i in range(len(data)-time_step-1):
    X.append(data[i:(i+time_step), 0])
    y.append(data[i+time_step, 0])
```

```
X = np.array(X)
y = np.array(y)
```

```
X = X.reshape(X.shape[0], X.shape[1], 1)
```

```
# Step 6: Split Data
```

```
train_size = int(len(X)*0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
```

```
# Step 7: Build Model
```

```
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(time_step,1)))
model.add(LSTM(50))
model.add(Dense(1))
```

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

```
# Step 8: Train
```

```
history = model.fit(X_train, y_train, epochs=10,
                    validation_data=(X_test, y_test))
```

```
# Step 9: Prediction
```

```
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
```

```
# Step 10: Inverse Scaling
```

```
train_pred = scaler.inverse_transform(train_pred)
test_pred = scaler.inverse_transform(test_pred)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To remember the important tasks and forget the less important tasks using LSTM	
Experiment No: 05	Date:	Enrolment No: 92410133023

Step 11: Plot

Create full-size arrays filled with NaN

```
train_plot = np.empty((len(data), 1))
train_plot[:] = np.nan
```

```
test_plot = np.empty((len(data), 1))
test_plot[:] = np.nan
```

Place train predictions

```
train_plot[time_step:time_step+len(train_pred)] = train_pred
```

Place test predictions

```
test_start = time_step + len(train_pred)
test_plot[test_start:test_start+len(test_pred)] = test_pred
```

Plot everything

```
plt.figure(figsize=(10,5))
plt.plot(scaler.inverse_transform(data), label="Actual")
plt.plot(train_plot, label="Train Prediction")
plt.plot(test_plot, label="Test Prediction")
```

```
plt.legend()
plt.title("TCS Stock Prediction")
plt.show()
```

Output:

